

Class 2: Perceptron Model

Learning Objectives

After this class, you will be able to:

- Understand the concept and working of a Perceptron.
- Explain the Perceptron learning algorithm.
- Solve a Perceptron example step-by-step.
- Identify the advantages and limitations of the Perceptron model.

1 1. Introduction

- The **Perceptron** is the simplest form of an Artificial Neural Network (ANN).
- It was introduced by **Frank Rosenblatt** in 1958.
- It is a **binary classifier** — used to classify input data into two categories (for example: 0 or 1, Yes or No, Positive or Negative).

2 2. Structure of a Perceptron

- A perceptron consists of:
 1. Input layer — input signals $x_1, x_2, \dots, x_n$.
 2. Weights $w_1, w_2, \dots, w_n$ associated with each input.
 3. A bias term b .
 4. An activation function (usually a **step function**).

Mathematical Model:

$$y_{in} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Activation Function (Step Function):

$$f(y_{in}) = \begin{cases} 1, & \text{if } y_{in} > 0 \\ 0, & \text{if } y_{in} \leq 0 \end{cases}$$

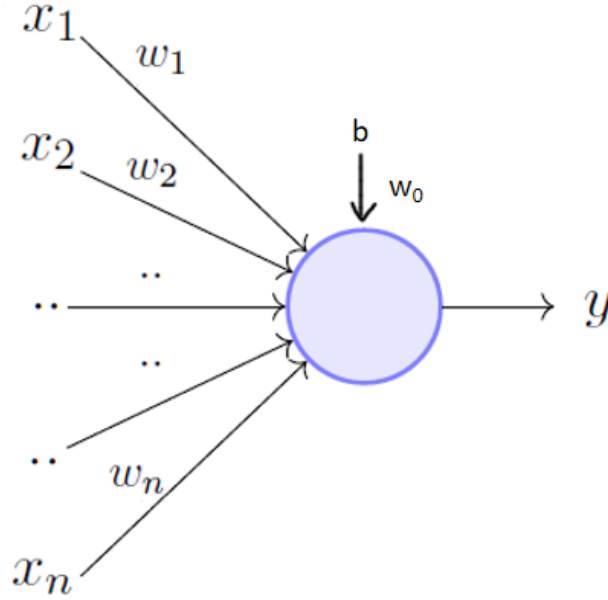


Figure 1: **Structure of a Single-Layer Perceptron.** Inputs are multiplied by weights, summed with a bias, and passed through a step activation function to produce a binary output.

3. Perceptron Learning Algorithm

The perceptron learning algorithm is used to find suitable weights and bias values so that the perceptron can correctly classify the training data.

Algorithm Steps:

1. Initialize weights $w_1, w_2, \dots, w_n$ and bias b with small random values (often 0).
2. For each training sample $(x_1, x_2, \dots, x_n)$ with target output t :

- (a) Compute the net input:

$$y_{in} = \sum_{i=1}^n w_i x_i + b$$

- (b) Determine the output using the activation function:

$$y = f(y_{in})$$

- (c) Update the weights and bias if the output is wrong:

$$w_i(new) = w_i(old) + \Delta w_i$$

where

$$\Delta w_i = \eta(t - y)x_i$$

and η is the learning rate (a small constant like 0.1).

$$b(new) = b(old) + \eta(t - y)$$

3. Repeat the process for all training samples until the output is correct for every sample or a maximum number of iterations is reached.

4 4. Worked Example (Step-by-Step)

Problem Statement

Train a perceptron to implement the logical **AND** function using the following data:

x_1	x_2	Target t
0	0	0
0	1	0
1	0	0
1	1	1

Assume: Initial weights $w_1 = 0$, $w_2 = 0$, bias $b = 0$, and learning rate $\eta = 1$.

Step 1: Iteration 1 (Epoch 1)

Training Sample 1: $(x_1, x_2) = (0, 0), t = 0$

$$y_{in} = (0)(0) + (0)(0) + 0 = 0$$

$$y = f(0) = 0$$

$$\text{Error } (t - y) = 0 - 0 = 0$$

No change in weights or bias.

$$w_1 = 0, \quad w_2 = 0, \quad b = 0$$

Training Sample 2: $(x_1, x_2) = (0, 1), t = 0$

$$y_{in} = (0)(0) + (0)(1) + 0 = 0$$

$$y = 0, \quad (t - y) = 0$$

No update.

Training Sample 3: $(x_1, x_2) = (1, 0), t = 0$

$$y_{in} = (0)(1) + (0)(0) + 0 = 0$$

$$y = 0, \quad (t - y) = 0$$

No update.

Training Sample 4: $(x_1, x_2) = (1, 1), t = 1$

$$y_{in} = (0)(1) + (0)(1) + 0 = 0$$

$$y = 0$$

$$\text{Error } (t - y) = 1 - 0 = 1$$

$$\Delta w_1 = 1(1)(1) = 1, \quad \Delta w_2 = 1(1)(1) = 1, \quad \Delta b = 1(1) = 1$$

$$w_1 = 0 + 1 = 1, \quad w_2 = 0 + 1 = 1, \quad b = 0 + 1 = 1$$

Step 2: Iteration 2 (Epoch 2)

Training Sample 1: $(0, 0), t = 0$

$$y_{in} = (1)(0) + (1)(0) + 1 = 1$$

$$y = f(1) = 1$$

$$\text{Error } (t - y) = 0 - 1 = -1$$

$$\Delta w_1 = 1(-1)(0) = 0, \quad \Delta w_2 = 1(-1)(0) = 0, \quad \Delta b = 1(-1) = -1$$

$$b = 1 - 1 = 0$$

Training Sample 2: $(0, 1), t = 0$

$$y_{in} = (1)(0) + (1)(1) + 0 = 1$$

$$y = 1, \quad (t - y) = -1$$

$$\Delta w_1 = 0, \quad \Delta w_2 = -1, \quad \Delta b = -1$$

$$w_1 = 1, \quad w_2 = 0, \quad b = -1$$

Training Sample 3: $(1, 0), t = 0$

$$y_{in} = (1)(1) + (0)(0) - 1 = 0$$

$$y = 0, \quad (t - y) = 0$$

No update.

Training Sample 4: $(1, 1), t = 1$

$$y_{in} = (1)(1) + (0)(1) - 1 = 0$$

$$y = 0, \quad (t - y) = 1$$

$$\Delta w_1 = 1(1)(1) = 1, \quad \Delta w_2 = 1(1)(1) = 1, \quad \Delta b = 1$$

$$w_1 = 2, \quad w_2 = 1, \quad b = 0$$

Step 3: Iteration 3 (Epoch 3)

Now check all samples again with updated weights:

- (0,0): $y_{in} = 0 \Rightarrow y = 0$
- (0,1): $y_{in} = 1 \Rightarrow y = 1$ (should be 0)

Continue training until all outputs match targets. After several epochs, the weights converge to values that correctly classify all samples:

$$w_1 = 1, \quad w_2 = 1, \quad b = -1.5$$

—

Final Decision Rule

$$y = \begin{cases} 1, & \text{if } (x_1 + x_2 - 1.5) > 0 \\ 0, & \text{otherwise} \end{cases}$$

This implements the AND logic perfectly.

5. Advantages of the Perceptron

- Simple and easy to implement.
- Useful for linearly separable problems (like AND, OR).
- Forms the foundation for more complex neural networks.

6. Limitations of the Perceptron

- Cannot solve non-linearly separable problems (like XOR).
- Learning process may not converge if data is noisy.
- Output is binary — not suitable for continuous prediction.

Class Summary

- A perceptron is a basic neural model used for binary classification.
- It learns by adjusting weights based on errors in prediction.
- The perceptron can represent only linearly separable decision boundaries.

Next Class Preview

In the next class, we will study different **Activation Functions** used in neural networks such as Step, Sigmoid, Tanh, and ReLU — along with their mathematical forms and graphical interpretations.